

The Art Catalogue is a public API that provides access to a collection of artworks and their creators. It allows users to browse, search, and retrieve detailed information about art pieces. Administrative users can manage the catalogue (add, update, delete entries).

Entities

Artwork - An art piece with title, artist, date, medium, dimensions, and image URL.

Artist - Creator of artworks – name, birth/death years, nationality, biography.

User - Administrator with credentials for managing the catalogue. Regular users are anonymous.

Functional Requirements

- Retrieve a paginated, filterable, and searchable list of artworks.
- Filter artworks by artist name, year range, or medium.
- Get detailed information about a specific artwork, including its artist.
- Retrieve a list of all artists (paginated) with basic info.
- Get a detailed artist profile (including a list of their works).
- Administrative operations (authenticated):
 - Add a new artwork.
 - Update existing artwork details.
 - Delete an artwork.
 - Add/edit/delete artists.
- All responses include HATEOAS links for navigation and related resources.

Non-Functional Requirements

- All responses are in JSON format.
- Public endpoints (read-only) do not require authentication.
- Write/update/delete operations require a valid JWT token with admin role.
- Passwords for admin users are hashed.
- All API calls must be over HTTPS.

- Pagination: every collection endpoint supports `page` (default 1) and `limit` (default 20, max 100).
- Caching: all public GET endpoints are cached with appropriate `Cache-Control` headers. Admin operations invalidate relevant caches.
- Error responses follow a standard structure: `{ "timestamp": "...", "status": 400, "error": "Bad Request", "message": "...", "path": "..." }`.
- Response times: 95th percentile < 300 ms for cached endpoints.

Data Models

Artwork

```
{  
  "id": "uuid",  
  "title": "Starry Night",  
  "artist": {  
    "id": "uuid",  
    "name": "Vincent van Gogh"  
  },  
  "date": "1889",  
  "medium": "Oil on canvas",  
  "dimensions": "73.7 × 92.1 cm",  
  "imageUrl": "https://...",  
  "description": "...",  
  "_links": {  
    "self": { "href": "/api/artworks/{id}" },  
    "artist": { "href": "/api/artists/{artistId}" }  
  }  
}
```

Artist

```
{
```

```

    "id": "uuid",
    "name": "Vincent van Gogh",
    "birthYear": 1853,
    "deathYear": 1890,
    "nationality": "Dutch",
    "biography": "...",
    "_links": {
      "self": { "href": "/api/artists/{id}" },
      "works": { "href": "/api/artworks?artistId={id}" }
    }
  }
}

```

User (admin)

```

```json
{
 "id": "uuid",
 "username": "admin"
}

```

## API Endpoints

### Public Endpoints (no authentication)

GET	`/api/artworks`	List artworks with filtering, search, pagination.
GET	`/api/artworks/{id}`	Get full artwork details.
GET	`/api/artists`	List all artists (paginated).
GET	`/api/artists/{id}`	Get artist profile with links to their works.

### Filters for GET /api/artworks:

- `q` – search in title, artist name, description (string)

- `artistId` – filter by artist UUID
- `yearFrom`, `yearTo` – date range (integer)
- `medium` – exact match (string)
- `page` – default 1
- `limit` – default 20, max 100

Response (200 OK):

```
{
 "items": [{ ...artwork objects... }],
 "page": 1,
 "limit": 20,
 "totalItems": 150,
 "totalPages": 8,
 "_links": {
 "self": { "href": "/api/artworks?page=1&limit=20" },
 "next": { "href": "/api/artworks?page=2&limit=20" },
 "last": { "href": "/api/artworks?page=8&limit=20" },
 "artists": { "href": "/api/artists" }
 }
}
```

Error: 400 for invalid filters, 404 if artwork not found.

## Authentication

| POST | `/api/auth/login` | Admin login – returns JWT access token.

- Access token valid for 1 hour.

## Administrative Endpoints (require JWT with admin role)

| POST | `/api/artworks` | Create a new artwork.

PUT	`/api/artworks/{id}`	Fully update an artwork (all fields required).
PATCH	`/api/artworks/{id}`	Partially update an artwork.
DELETE	`/api/artworks/{id}`	Delete an artwork.
POST	`/api/artists`	Create a new artist.
PUT	`/api/artists/{id}`	Update artist.
DELETE	`/api/artists/{id}`	Delete artist (if no artworks reference it).

### Status Codes – Mapped to Descriptions

200	OK – successful GET, PUT, PATCH.
201	Created – successful POST with Location header.
204	No Content – successful DELETE.
400	Bad Request – malformed request, validation errors.
401	Unauthorized – missing or invalid JWT token.
403	Forbidden – authenticated but not admin.
404	Not Found – resource does not exist.
409	Conflict – e.g., deletion prevented by foreign key constraint.
500	Internal Server Error – unexpected condition.

### Richardson Maturity Model (Level 3 – HATEOAS)

- Every resource representation includes `\_links` to related resources (self, artist).
- Collection responses include `\_links` for pagination (next, prev, first, last) and top-level navigation.
- POST responses include `Location` header and the created resource with `\_links.self`.
- This enables clients to discover actions dynamically without hardcoding URLs.

### Authentication – JWT Structure

JWT payload:

```
{
 "sub": "admin-user-id",
```

```
"username": "admin",
"role": "admin",
"iat": 1234567890,
"exp": 1234571490
}
```

- Access token: short-lived (1 hour), sent in `Authorization: Bearer <token>`.
- Refresh token: optional, can be implemented if needed.

Authentication errors:

- Missing token → `401` with `WWW-Authenticate: Bearer`.
- Invalid/expired → `401` with error message.
- Valid but not admin → `403 Forbidden`.

## Pagination

All collection endpoints (`/api/artworks`, `/api/artists`) accept:

- `page` – 1-indexed (default 1).
- `limit` – items per page (default 20, max 100).

Response includes:

```
{
 "items": [...],
 "page": 1,
 "limit": 20,
 "totalItems": 150,
 "totalPages": 8,
 "_links": {
 "self": { "href": "..."} },
}
```

```

 "next": { "href": "..."},
 "prev": { "href": "..."},
 "first": { "href": "..."},
 "last": { "href": "..."}
 }
}

```

## Caching Strategy

| `GET /artworks` | `Cache-Control: public, max-age=3600` (1 hour) | Public catalogue, changes infrequently.

| `GET /artworks/{id}` | `Cache-Control: public, max-age=3600` | Artwork details are static.

| `GET /artists` | `Cache-Control: public, max-age=86400` (24h) | Artist data rarely changes. |

| `POST`, `PUT`, `PATCH`, `DELETE` | no cache (and invalidate relevant caches) | State-changing operations must be fresh.

Cache invalidation: after any admin write operation, the server should purge the cache for the affected resource collections (e.g., after adding an artwork, the `/api/artworks` cache is invalidated).

## Example: Complete Workflow

Anonymous user:

1. GET `/api/artworks?q=monet&page=2` → receives paginated list with links.
2. GET `/api/artworks/{id}` → obtains full details with artist link.

Admin user:

1. Login: POST `/api/auth/login` → receives JWT.
2. Add new artwork: POST `/api/artworks` with token → receives 201 and new artwork.
3. Update artwork: PATCH `/api/artworks/{id}` → 200 OK.
4. Delete artwork: DELETE `/api/artworks/{id}` → 204 No Content.